Personal Statement

Yanghao Chen

My first research project at Tongji University involved fine-tuning a language model for mechanics of materials under Professor Ying Zhao's guidance. I initially relied on public datasets and Colab notebooks, but the mechanics content was limited and the data format did not match the training code. I examined how the examples were structured, prepared my own instruction and response pairs, and adapted the notebook to load and process them. Tracing how the data were read and processed helped me understand the training workflow. I gradually learned to break the task into specific questions about data formats, loading, and training settings, then consult resources and check the effect of each change.

During the China International College Students' Innovation Competition, my teammates and I needed to deploy our fine-tuned model locally, but the available GPU memory was insufficient. After discussing possible solutions, we reduced numerical precision to lower memory requirements and adjusted how the model was saved and loaded, eventually deploying it locally. Working through these difficulties together taught me to explain what I had tried and where I needed help. Our team went on to receive a national-level award in the competition. Having learned from public datasets and notebooks myself, I also wanted to provide resources that others could learn from and use, so I shared my datasets and fine-tuned models on Hugging Face.

Language model fine-tuning made me want to understand neural networks and their use in mechanics more deeply. Continuing under Professor Ying Zhao's guidance, I studied conditional ConvLSTM models for data-driven prediction of lithium concentration and von Mises stress fields in battery materials. Using MATLAB scripts to run COMSOL simulations, I generated field images in time order and paired each sequence with grain-orientation and C-rate maps. During autoregressive prediction, errors accumulated and images became distorted as predicted frames became inputs to later steps. I introduced scheduled sampling to gradually include predicted frames during training, then compared multi-step predictions with COMSOL reference sequences. For the sequences examined, the revised training produced lower image errors over the same prediction horizon and delayed visible distortion compared with training without scheduled sampling. I learned to assess the model over successive prediction steps and to make training reflect how it would actually be used. Preparing simulation labels also raised a question: could physical laws guide neural network training without precomputed reference solutions?

I explored this question under Professor Xianyang (Tom) Chen's guidance, using physics-informed neural networks (PINNs) for the one-dimensional inviscid Burgers equation. Using automatic differentiation in PyTorch, I computed the partial differential equation (PDE) residual and trained the network with residual, initial-condition, and boundary-condition losses, without precomputed solution labels. Capturing the sharp shock transition was difficult, so I added an artificial-viscosity term with a fixed coefficient to the residual. Against the analytical inviscid solution, this model had lower relative L^2 error but higher maximum pointwise error than the standard PINN. I examined shock location and transition width to understand the difference. Artificial viscosity also changed the equation, so lower error alone did not establish a better method. My PINN was trained for fixed initial and boundary conditions; changing them

would require retraining. I wondered whether a network could learn a mapping from conditions to solutions and handle different inputs.

This question led me to neural operators, which learn function-to-function mappings and handle different inputs within a class of problems without retraining. At Johns Hopkins University, under Professor Somdatta Goswami's supervision, I helped develop a hybrid finite element–neural operator solver for solid mechanics. My collaborator and I initially tried predicting displacements and computing stresses from them, but this amplified prediction errors. We regularly discussed our attempts and decided to predict stresses directly. I generated data with GPU-accelerated JAX-FEM and trained Transolver in JAX to map interface displacement fields to subdomain stress fields. I coupled the operator with FEM through non-overlapping domain decomposition. However, errors were small on held-out Gaussian random field (GRF) samples but much larger on actual coupling inputs. Comparing the two sets of displacements revealed a distribution shift. I adjusted the GRF length scale and amplitude, generated new data, and retrained the model. After retraining, the model maintained good stress-prediction accuracy on the GRF validation set and showed lower errors for coupling-generated inputs in cylinder and bracket tests. Because predicted stresses determine the interface forces used in subsequent displacement updates, I evaluated the coupled solver through stress accuracy, interface displacement compatibility, force balance, and convergence. I am now extending this work to three-dimensional fiber-reinforced composites, using FEM for the matrix and sharing neural-operator weights across fiber subdomains.

I want to pursue a PhD to further study how machine learning and numerical methods can be combined for solid mechanics. My projects have given me experience implementing and testing methods. They have also shown me that interpreting results and judging a method's applicability require a stronger foundation in mechanics and numerical analysis. Working with JAX and JAX-FEM has drawn me to GPU-accelerated scientific computing and differentiable programming. I hope to learn more about these methods and explore how neural operators and numerical solvers can work together within differentiable computational graphs. I also want to understand how the resulting gradients can support structural optimization and inverse problems. In a doctoral group, I would like to share my ideas and difficulties openly, learn from collaborators, and provide code and data that others can use.